

autolisp-benchmark — Specifications

clautolisp

July 2, 2026

Contents

1	Scope	1
2	Portability contract	1
3	The clock	1
3.1	clautolisp requirement	2
4	Workload contract	2
5	Calibration and the run loop	3
6	Metrics	3
7	Operation classes	3
8	Detection	4
9	Non-goals (this release)	4

1 Scope

This document specifies the timing model of the benchmark harness and the contract every workload must satisfy. It is normative for the code under `harness/` and `benchmarks/`. The prose walkthrough of **running** the suite is in the user manual; the design rationale is in the development plan.

2 Portability contract

- The harness and all workloads are written in the common AutoLISP / Visual LISP subset available on clautolisp, AutoCAD, and BricsCAD.
- No dependency on Common Lisp, Express Tools, DOSLib, or COM/ActiveX.
- CAD workloads use entity primitives only (`entmake`, `entlast`, `entget`, `entmod`, `entdel`). (`command ...`) and `vl-cmdf` are forbidden: the former is unimplemented on clautolisp, the latter is a no-op there.
- The suite must load and run identically on every target; only the **numbers** differ.

3 The clock

The harness measures elapsed wall-clock time with a millisecond clock abstracted by `bench-clock-ms`, chosen once at startup by `bench-timer-init`:

1. Read (`getvar "MILLISECS"`), spin briefly, read again. If it advanced, use `MILLISECS` (integer milliseconds; native on AutoCAD/BricsCAD, live on clautolisp in the 1.3.x line).
2. Otherwise use (`getvar "DATE"`) — a real Julian date whose fractional part is the fraction of the day — multiplied by 86 400 000 to yield milliseconds.

Only **differences** between two readings are ever used, so the absolute epoch is irrelevant and the two sources are interchangeable for timing.

3.1 clautolisp requirement

On clautolisp the timing system variables `MILLISECS`, `DATE`, `CDATE`, `TDUSRTIMER`, and `TDINDWG` must be **live** (sourced from the runtime clocks), not the static zero cells of the raw sysvar catalogue. This is provided by clautolisp in the 1.3.x line (see the clautolisp release notes, "Live timing system variables"). Against an older clautolisp the clock would read a constant and every elapsed interval would be zero.

4 Workload contract

A workload is a function (`bench-NAME reps`) where:

- `reps` is a positive integer.
- The function performs exactly `reps` units of the operation class in a tight loop. One "unit" is a fixed bundle of operations chosen to be representative of the class.
- One-time setup (allocating a constant table, computing a temp-file path) happens once, outside the `reps` loop.
- The return value is unspecified and ignored, but the function must actually consume its intermediate results within the loop so an optimising evaluator cannot elide the work. (AutoLISP interpreters do not perform dead-code elimination, but the workloads are written to hold under a hypothetically compiling engine too.)

Registration: each workload file ends with

```
(defbench "NAME" "CLASS" 'bench-NAME)
```

which appends (NAME CLASS FUNCTION-SYMBOL) to `*benchmarks*`.

5 Calibration and the run loop

For each workload the engine:

1. **Calibrates** a batch size: starting at 1 and growing geometrically (x4), it finds the smallest batch whose single evaluation takes at least `*bench-calibrate-ms*` milliseconds (default 50), capped at 2e8. This amortises per-call dispatch overhead and timer granularity.
2. **Runs** the workload with that batch repeatedly, summing iterations, until the accumulated elapsed time reaches `*bench-seconds* * 1000` milliseconds (default 5 s).
3. **Records** an alist ((name) (category) (reps . N) (ms . T) (batch . B)).

6 Metrics

Given `reps` iterations completed in `ms` milliseconds:

- $\text{iterations/second} = \text{reps} * 1000 / \text{ms}$
- $\text{microseconds/iteration} = \text{ms} * 1000 / \text{reps}$

The overall summary sums `reps` and `ms` across classes and reports the aggregate iterations/second. Because each class is time-boxed to the same budget, the elapsed column is approximately constant; throughput is the comparison axis.

7 Operation classes

Exactly one workload per class in this release:

Class	Unit of work (per iteration)
arithmetic	a bundle of int/float ops, <code>sqrt/=expt/=sin/=cos</code> , <code>rem</code> , <code>abs</code> , <code>min/=max</code>
lists	build an ~12-element list; <code>cons</code> , <code>append</code> , <code>reverse</code> , <code>mapcar</code> , <code>nth</code> , <code>member</code> , <code>assoc</code> , <code>length</code>
strings	build a string; <code>strlen</code> , <code>substr</code> , <code>strcase up/down</code> , <code>atoi/atof</code> , <code>wcmatch</code> , <code>vl-string-search</code>
serialization	build a nested mixed structure; <code>vl-prin1-to-string</code> then <code>read</code>
file-io	write 16 lines to the scratch file, <code>close</code> , <code>reopen</code> , read all lines, <code>close</code>
entity	entmake a LINE; <code>entlast</code> ; <code>entget</code> ; subst the end-point; <code>entmod</code> ; <code>entdel</code>
memory-gc	build and drop ~200 cons cells with nested lists+strings; (<code>gc</code>) every <code>*bench-gc-interval</code>

8 Detection

- Implementation identity: (`ver`) (a string). The mock host's PRODUCT sysvar reads "BricsCAD" even under clautolisp, so PRODUCT is **not** a valid discriminator; (`ver`) is.
- Platform: the OS / OSTYPE environment variables when present, else (`getvar "PLATFORM"`).

9 Non-goals (this release)

- Machine-readable result files and an automatic cross-engine diff (planned; see PLAN.md).

- A (`command ...`)-based CAD workload (blocked on clautolisp `COMMAND`).
- Interpreted-vs-compiled (`.fas`) and `vlisp-compile` optimisation levels (CAD-only; clautolisp has no compile step).